

龙贝格积分法上机报告

周才翔

数试2402 2243711412

2026 年 5 月 5 日

1 龙贝格积分法原理

1.1 引言

龙贝格积分法是利用梯形变步长积分法将积分区间逐次分半计算，用低精度公式逐次适当组合而获得较高计算精度的一种外推法。它无需预先估计被积函数的高阶导数最大值，通过自动调整步长和外推过程满足精度要求，具有计算简单、数值稳定性好、算法易于实现等优点，是工程和科学计算中最常用的高精度数值积分方法之一。

1.2 龙贝格积分公式推导

对于复化梯形公式而言，可取积分的近似值为

$$I[f] \approx T_{2n} + \frac{1}{4-1} (T_{2n} - T_n) = \bar{T}_{2n}. \quad (1)$$

可以证明，式(1)恰好是复化辛普森求积公式，即

$$S_n = T_{2n} + \frac{1}{4-1} (T_{2n} - T_n). \quad (3)$$

同理，对复化辛普森求积公式有

$$I[f] - S_n = -\frac{b-a}{2880} h^4 f^{(4)}(\eta), \quad a \leq \eta \leq b,$$
$$I[f] - S_{2n} = -\frac{b-a}{2880} \left(\frac{h}{2}\right)^4 f^{(4)}(\eta_1), \quad a \leq \eta_1 \leq b.$$

假定 $f^{(4)}(x)$ 在区间 $[a, b]$ 上连续且变化不大，可以认为 $f^{(4)}(\eta) \approx f^{(4)}(\eta_1)$ 。将以上两式相除得

$$\frac{I[f] - S_n}{I[f] - S_{2n}} \approx 16,$$

由此解得

$$I[f] \approx S_{2n} + \frac{1}{4^2 - 1} (S_{2n} - S_n) \triangleq \bar{S}_{2n}, \quad (2)$$

可以证明, 式(2)恰好是复化科茨求积公式, 即

$$C_n = S_{2n} + \frac{1}{4^2 - 1} (S_{2n} - S_n). \quad (4)$$

同理, 对复化科茨求积公式有

$$R_n = C_{2n} + \frac{1}{4^3 - 1} (C_{2n} - C_n), \quad (5)$$

式(5)称为龙贝格积分公式。显然, 式(3)—(5)的计算最终归结为复化梯形求积公式的计算。

为减少计算量, 在计算 T_{2n} 时, 可以利用 T_n 的结果, 其递推公式为

$$T_{2^{k+1}} = \frac{1}{2} T_{2^k} + \frac{b-a}{2^{k+1}} \sum_{i=1}^{2^k} f\left(a + (2i-1) \frac{b-a}{2^{k+1}}\right), \quad k = 0, 1, 2, \dots \quad (6)$$

开始计算时取

$$T_1 = \frac{b-a}{2} [f(a) + f(b)].$$

1.3 龙贝格积分法的计算流程

龙贝格积分法的计算过程按教材图6.1所示的逐行递推顺序进行:

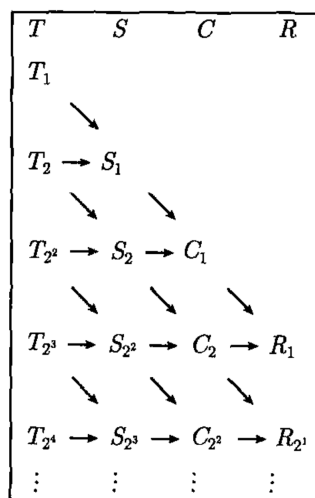


图 1: 龙贝格积分法流程图

1. 第1行: 计算初始梯形值 T_1
2. 第2行: 计算 T_2 , 外推得辛普森值 S_1
3. 第3行: 计算 T_4 , 外推得辛普森值 S_2 、科茨值 C_1
4. 第4行: 计算 T_8 , 外推得辛普森值 S_4 、科茨值 C_2 、龙贝格值 R_1

5. 第5行：计算 T_{16} ，外推得辛普森值 S_8 、科茨值 C_4 、龙贝格值 R_2
6. 以此类推，直到相邻两个龙贝格值的差 $|R_{2^{k+1}} - R_{2^k}| < \varepsilon$ ，则取 $I[f] \approx R_{2^{k+1}}$ ；否则，继续计算，直到满足精度要求或达到最大迭代次数为止。

2 Python编程实现

```
import math

def romberg_integral(f, a, b, eps=1e-12, max_iter=20):
    romberg_table = []
    h = b - a

    row0 = [0.5 * h * (f(a) + f(b))]
    romberg_table.append(row0)

    for k in range(1, max_iter):
        h /= 2
        sum_f = 0.0
        for i in range(1, 2**k + 1, 2):
            sum_f += f(a + i * h)
        t_k = 0.5 * romberg_table[k-1][0] + h * sum_f
        current_row = [t_k]

        for m in range(1, k + 1):
            coeff = 4 ** m
            extrap_val = (coeff * current_row[m-1]
                          - romberg_table[k-1][m-1]) / (coeff - 1)
            current_row.append(extrap_val)

        romberg_table.append(current_row)

    if k >= 3:
        if len(current_row) >= 4 and len(romberg_table[k-1]) >= 4:
            if abs(current_row[3] - romberg_table[k-1][3]) < eps:
                result = current_row[3]
                return result, romberg_table
```

```

    print(f"\n达到最大迭代次数{max_iter}, 返回当前最优值")
    last_row = romberg_table[-1]
    result = last_row[min(3, len(last_row)-1)]
    return result, romberg_table

def print_romberg_table(romberg_table, precision=12):

    print("龙贝格积分计算表")
    header = f"{'k':<2} {'T_{2^k}':<{precision+8}}"
    {'S_{2^k}':<{precision+8}} {'C_{2^k}':<{precision+8}}
    {'R_{2^k}':<{precision+8}}"
    print(header)

    for k in range(len(romberg_table)):
        row = romberg_table[k]
        t_val = f"{row[0]:.{precision}f}" if len(row)>=1 else ""
        s_val = f"{row[1]:.{precision}f}" if len(row)>=2 else ""
        c_val = f"{row[2]:.{precision}f}" if len(row)>=3 else ""
        r_val = f"{row[3]:.{precision}f}" if len(row)>=4 else ""
        print(f"{k:<2} {t_val:<{precision+8}} {s_val:<{precision+8}}
              {c_val:<{precision+8}} {r_val:<{precision+8}}")

if __name__ == "__main__":
    print("龙贝格积分法")
    print("输入提示: ")
    print("1. 三角函数: sin(x), cos(x), tan(x)")
    print("2. 指对函数: exp(x), log(x)")
    print("3. 幂函数: x**2, x**0.5")
    print("4. 开方: sqrt(x)")
    print("5. 分式: sin(x)/x")
    print("6. 常数: pi, e")

    while True:
        try:
            func_str = input("\n请输入被积函数 f(x) = ")
            safe_globals = {
                'sin': math.sin, 'cos': math.cos, 'tan': math.tan,

```

```
        'exp': math.exp, 'log': math.log, 'sqrt': math.sqrt,
        'abs': abs, 'pi': math.pi, 'e': math.e
    }
    def f(x):
        if abs(x) < 1e-15:
            return eval(func_str, safe_globals, {'x': 1e-15})
        else:
            return eval(func_str, safe_globals, {'x': x})
    f(1.0)
    break
except Exception as e:
    print(f"输入错误: {e}, 请检查语法并重新输入")

while True:
    try:
        a = float(input("请输入积分下限 a = "))
        b = float(input("请输入积分上限 b = "))
        break
    except Exception as e:
        print(f"输入错误: {e}, 请输入数字")

while True:
    try:
        eps_input = input("请输入精度要求 (默认1e-10, 直接回车使用默认值): ")
        eps = float(eps_input) if eps_input.strip() else 1e-10
        if eps <= 0:
            print("精度必须大于0")
            continue
        break
    except Exception as e:
        print(f"输入错误: {e}, 请输入正数")

result, table = romberg_integral(f, a, b, eps)

print_romberg_table(table)
print(f"\n积分近似值: {result:.12f}")
```

3 实例计算与结果分析

3.1 教材例题6.1.2验证

3.1.1 算例描述

选取教材例6.1.2的经典积分：

$$I = \int_0^1 \frac{\sin x}{x} dx$$

教材要求精度 10^{-12} ，精确参考值为 0.946083070367。

3.1.2 输入参数

请输入被积函数 $f(x) = \sin(x)/x$

请输入积分下限 $a = 0$

请输入积分上限 $b = 1$

请输入精度要求（默认 $1e-10$ ，直接回车使用默认值）： $1e-12$

3.1.3 计算结果

表 1: 教材例题6.1.2龙贝格积分计算表

k	T_{2^k}	S_{2^k}	C_{2^k}	R_{2^k}
0	0.920735492404			
1	0.939793284806	0.946145882274		
2	0.944513521665	0.946086933952	0.946083004064	
3	0.945690863583	0.946083310888	0.946083069351	0.946083070387
4	0.945985029934	0.946083085385	0.946083070351	0.946083070367
5	0.946058560963	0.946083071306	0.946083070367	0.946083070367

积分近似值：0.946083070367

3.1.4 结果分析

迭代到第5行（ $k = 5$ ）时，相邻龙贝格值的差小于 10^{-12} ，满足精度要求。远快于复化梯形法。

3.2 教材习题6.6测试

3.2.1 算例描述

选取教材习题6.6的积分：

$$I = \int_0^1 \frac{1}{1+x^2} dx$$

该积分的精确值为 $\frac{\pi}{4} \approx 0.785398163397$ ，教材要求误差不超过 0.5×10^{-5} 。

3.2.2 输入参数

请输入被积函数 $f(x) = 1/(1+x**2)$

请输入积分下限 $a = 0$

请输入积分上限 $b = 1$

请输入精度要求（默认1e-10，直接回车使用默认值）：0.5e-5

3.2.3 计算结果

表 2: 教材习题6.6龙贝格积分计算表

k	T_{2^k}	S_{2^k}	C_{2^k}	R_{2^k}
0	0.750000000000			
1	0.775000000000	0.783333333333		
2	0.782794117647	0.785392156863	0.785529411765	
3	0.784747123623	0.785398125615	0.785398523531	0.785396445940
4	0.785235403010	0.785398162806	0.785398165286	0.785398159599

积分近似值：0.785398159599

3.2.4 结果分析

迭代到第4行（ $k = 4$ ）时，相邻龙贝格值的差小于 0.5×10^{-5} ，满足精度要求。

4 总结

本报告简述了龙贝格积分法的核心公式，实现了通用的交互式龙贝格积分程序，并通过教材例题和习题进行了验证。结果表明，龙贝格积分法在少量计算量下即可获得极高的积分精度，收敛速度远快于复化梯形法和复化辛普森法。实现的程序具有良好的交互性和稳定性，能够处理常见的被积函数和特殊点问题，计算结果与教材一致。

龙贝格积分法兼顾了精度、效率和易用性，是数值积分领域的经典方法，在科学计算和工程应用中具有广泛的实用价值。